

NPLify · P0 Technical Baseline

Model Gaps & Proposals — candidate changes for ERD v5.3 · Draft v1.0

New XP Technologies Limited

8 October 2026 · Internal

Status: Written against ERD & Data Model Draft v5.2. Sources: NPL's "Table of Remittance Customers Fees" (October 2026), the Fee Outlines v0.1 workbook as returned by Sud (NPL) on 8 October with its two new tabs "Project Context" and "Payment methods", and Ayush's clarifications of the same day on the Raen split and on rebates. **Nothing in this document is applied.** Each proposal is a self-contained change with its exact footprint; approved items become decisions D20 onward and ship together as v5.3 with the dependent documents regenerated.

Audience: New XP engineering and, for Section 4, NPL Management. Each proposal reads in three steps — what is happening in plain terms, why the current model cannot express it, the fix and how it resolves it — followed by the ERD footprint in the model's own names.

How to read the footprints. *Gains* means a new column on an existing table; *new table* adds a box to a diagram and one to the count (54 tables + 4 views in v5.2); *rule* is a write-time check; *invariant* is a standing statement the daily checks assert; *purpose* is a value in the chart of accounts. Diagram numbers refer to the ERD baseline (1a configuration, 1b partner configuration, 2 rates, 3a deal, 3b collection and settlement, 4 ledger).

1 · The thirteen proposals at a glance

#	Proposal	Source	Where it stands with NPL
A	A special sender gets a different split	Fee table (7 Oct)	Confirmed by NPL (8 Oct): the receiver part is fixed at 0.6 % for all Evo senders..
B	Fees that drop with volume	Fee table (7 Oct)	Rule confirmed by NPL (8 Oct): the USD 500,000 threshold is counted per calendar month, not a rolling 30 days; last month's GDC volume was 163,124.
C	The fee the customer sees isn't the fee that was charged	Fee table (7 Oct)	Confirmed by NPL (8 Oct): the "1 % shown" rule is for the BF project only and may later become "partner rate + markup".
D	NPL as its own exchange desk — NPL-GR as a project	Fee table (7 Oct)	NPL's suggestion (8 Oct): "technically NPL is the client — imagine a business unit NPL-GR"; Bala is the sender, NPL-GR the receiver.
E	Paying a EUR invoice in USDT, in pieces	Fee table (7 Oct)	Wanted by NPL (8 Oct): "good to have".

#	Proposal	Source	Where it stands with NPL
F	Rounding that isn't "drop the cents"	Fee table (7 Oct)	Confirmed by NPL (8 Oct): round up to the nearest 1,000 for Asian currencies (THB, INR, JPY ...), to the nearest unit for USD, SGD, HKD; round up the USDT a reseller customer pays "when possible" — deliberate, not a convenience..
G	NPL's markup shared with the customer who provides the sub-account — several parties, money held by the party	Sud's workbook and Ayush's clarifications (8 Oct)	New (8 Oct).
H	Rebates as a separate earnings stream — and statement lines for deals NPL never saw	Sud's workbook and Ayush's clarifications (8 Oct)	New (8 Oct).
I	Partner rates quoted the other way round	Sud's workbook and Ayush's clarifications (8 Oct)	New (8 Oct).
J	Bank cutoff, rate lock and next-day conversion	Sud's workbook and Ayush's clarifications (8 Oct)	New (8 Oct).
K	Cash by token, with rates that depend on the city	Sud's workbook and Ayush's clarifications (8 Oct)	New (8 Oct).
L	Sender wallet screening and risk-routed receiving wallets	Sud's workbook and Ayush's clarifications (8 Oct)	New (8 Oct).
M	One-time local bank accounts and the reversal of a collection	Sud's workbook and Ayush's clarifications (8 Oct)	New (8 Oct).

A–F were identified from the fee table and explained to NPL on 7 October; NPL's answers of 8 October confirm all six and settle their open parameters (noted under each). G–M are new: they come from the answers themselves and from the two tabs Sud added, which describe how NPL's projects and payment methods actually run. Section 3 lists the smaller facts from those tabs that the model already handles by configuration. Section 6 is the list of questions still open with NPL.

2 · The proposals

A · A special sender gets a different split

Where it stands. Confirmed by NPL (8 Oct): the receiver part is fixed at 0.6 % for all Evo senders.

WHAT IS HAPPENING

For most Evo customers the fee is 2 % in total: 1.4 % paid by the person sending the money, 0.6 % paid by Evo out of its own balance. For three favoured senders (GTSI, HG, Komodo) the sender pays only 1.0 % — but Evo still pays its 0.6 %. So the total drops to 1.6 %, and only the sender's half moved.

WHY THE MODEL CANNOT SAY IT

The fee rules are stored once per partner and currency pair (FEE_STRUCTURE), with the sender/receiver split baked in — 70/30 in Evo's case. The per-sender exception (FEE_OVERRIDE) only changes the total and keeps the same proportion: asked for 1.6 % it would charge the sender 1.12 % and Evo 0.48 %, wrong on both sides.

THE FIX, AND HOW IT CLOSSES THE GAP

Let the per-sender override carry its own two numbers — sender share and receiver share — instead of inheriting the proportion. The override for GTSI says "sender 1.0 %, receiver 0.6 %", the engine uses those directly, and the structure's 1.4 / 0.6 stays untouched for everyone else.

ERD FOOTPRINT

- FEE_OVERRIDE gains sender_share_pct and receiver_share_pct (decimal, optional). Both blank → the structure's proportion applies; both set → they must add up to pct. Enforced on save.
- DEAL gains fee_override_id (FK, nullable) so a leg records which override priced it — today that is only inferable.
- Diagrams 1b (FEE_OVERRIDE box) and 3a (one more stamped reference into DEAL). Dependent docs: Configuration Schema, Calculation Specification (the fee-split step reads the override's shares first).

B · Fees that drop with volume

Where it stands. Rule confirmed by NPL (8 Oct): the USD 500,000 threshold is counted per calendar month, not a rolling 30 days; last month's GDC volume was 163,124. Open: whether the lower tier applies to the deal that crosses the threshold or from the next one.

WHAT IS HAPPENING

GDC pays 2.5 % as long as it sends less than \$500K in a month; past that, every transaction is at 2.25 %. The price depends on how much they have sent lately.

WHY THE MODEL CANNOT SAY IT

A fee rule is a fixed percentage (or a range with a floor). Nothing in the model looks at the sender's recent volume when quoting. Today somebody has to notice GDC crossed \$500K and switch the fee by hand — and switch it back next month.

THE FIX, AND HOW IT CLOSSES THE GAP

A small tier table attached to a fee structure ("from \$0: 2.5 %; from \$500K: 2.25 %") and one rule: at quote time, add up what this client sent in the current calendar month — a figure the engine already has, since every collection is recorded — and pick the matching tier. The switch becomes automatic and auditable: the quote records which tier applied and the volume that justified it.

ERD FOOTPRINT

- New table FEE_TIER: id, fee_structure_id (FK), min_monthly_volume (decimal), volume_currency, pct, optional sender_share_pct / receiver_share_pct, effective_from. FEE_STRUCTURE owns FEE_TIER (1 : n); a structure with tiers must have one tier at volume 0.
- DEAL gains fee_tier_id (FK, nullable) and tier_volume_basis (decimal) — the tier applied and the calendar-month volume that justified it, stamped at quote.
- New invariant: the tier stamped on a leg is the one with the highest min_monthly_volume not exceeding the project's month-to-date Σ amount_in (in volume_currency) at quote time.
- New view SENDER_VOLUME(project, month) — a query over collections, not a stored field. Diagrams 1b (new box under FEE_STRUCTURE) and 3a. Count: 55 tables.

C · The fee the customer sees isn't the fee that was charged

Where it stands. Confirmed by NPL (8 Oct): the "1 % shown" rule is for the BF project only and may later become "partner rate + markup". The Raean sub-account quote uses the same recomputation (a "market" of 0.8650036 so that "4 %" shows).

WHAT IS HAPPENING

When Ali is the partner, the sender is told "fee: 1 %". In reality the total cost built into the deal is 1.4 % — Ali's 1 % is hidden inside the exchange rate it quotes, and NPL's 0.4 % sits on top. To make the maths on the customer's quote add up to "1 %", someone recomputes the "market rate" by hand so that the displayed rate $\times (1 - 1 \%)$ lands on the right euro amount.

WHY THE MODEL CANNOT SAY IT

The model stores the truth correctly — the real market snapshot, the partner's rate, the final sender rate, both fee parts. What it has no setting for is what to print on the quote: there is no field saying "show 1 %" when the recorded fee parts add to 1.4 %.

THE FIX, AND HOW IT CLOSES THE GAP

A presentation setting on the fee structure — a displayed fee percentage, or a rule such as "show the sender share on the partner-basis rate" — and the quote package derives the shown "source rate" from it, exactly as Operations does by hand today. The hand recomputation disappears, the customer sees the same figures as before, and the books still hold the real 1.4 %.

ERD FOOTPRINT

- FEE_STRUCTURE gains displayed_fee_basis (enum: actual / sender_share_on_partner_rate / fixed) and displayed_fee_pct (decimal, used when fixed).
- DEAL_GROUP gains displayed_fee_pct and displayed_source_rate (decimal), stamped at quote so a re-printed quote matches the original. Calc spec: $\text{displayed_source_rate} = \text{sender_rate} \div (1 - \text{displayed_fee_pct})$.
- Rule: displayed figures are derived from the true figures, never entered by hand; Operations sees the displayed pair, Finance sees both (the Roles matrix already draws this line). Diagrams 1b, 3a. Dependent docs: Calculation Specification (new step + an acceptance vector from the real Ali quote: market 0.86050, Ali $1 \div 1.1738$, shown 0.857097), Configuration Schema.

D · NPL as its own exchange desk — NPL-GR as a project

Where it stands. NPL's suggestion (8 Oct): "technically NPL is the client — imagine a business unit NPL-GR"; Bala is the sender, NPL-GR the receiver. Management still has to confirm the principle change.

WHAT IS HAPPENING

For the game-reseller business (Bala, Fair, 66 Group, TripleSeven, Alpha Plus, and 27 Group via THB cash) there is no outside partner for the conversion: the customer sends USDT straight to NPL's own wallet (LT Sub), NPL keeps its markup, and NPL pays the vendors in USDT on later days. NPL is holding and converting the money itself. Occasionally the USDT sits in Ali's wallet as NPL's own balance.

WHY THE MODEL CANNOT SAY IT

The model was built on a principle agreed early on: NPL never holds client money in the normal course — money sits at partners. Its own wallet exists only as an emergency route (a reroute), and invariant 3 says client money may be in NPL's wallet only under an approved reroute. A routine product that always goes through NPL's wallet breaks that rule on every deal, and there is no partner for the deal to hang its prices on.

THE FIX, AND HOW IT CLOSSES THE GAP

Take NPL's own framing: a project "NPL-GR" whose client is NPL's reseller business unit. Its partner configuration is an own desk — NPL's own wallet is that desk's collection endpoint, its pair is priced at market plus NPL's markup — and invariant 3 is narrowed to "client money may be in NPL's wallet only under an approved reroute or in an own-desk project". The custody view keeps showing every such balance and its age, which is what the principle was protecting. The alternative — keep the principle intact and run these flows outside NPLify — remains Management's to choose.

ERD FOOTPRINT

- PARTNER_CONFIG gains `is_own_desk` (bool, default false; enabling it is a Management-approved configuration change — new APPROVAL action `enable_own_desk`).
- COLLECTION_RECEIVING_ENDPOINT gains `own_wallet_id` (FK → OWN_WALLET, nullable) so the LT Sub wallet is both NPL's own wallet and the own desk's collection endpoint, without a duplicate address. USDT parked at Ali for NPL-GR is `CO.ALI.USDT.P00L`-style company money, not client money.
- Rates: an own-desk PARTNER_PAIR is priced `market_plus_pct` with NPL's markup; its PARTNER_RATE_VERSION rows are generated from the market snapshot rather than entered.
- Invariant 3 restated: `CL.OWN.*.HELD` may be debited only by a collection on a deal with an approved REROUTE or on a leg whose partner configuration is an own desk; the custody aging view reports both.
- Ledger: own-desk deals use the ordinary partner-coded accounts with the own-desk config as holder (`CL.OWNDESK.<CCY>.COLLECTED / DUE / INTRANSIT`); EXP_PARTNER is zero by construction. Diagrams 1a, 1b, 4. Dependent docs: Lifecycle (approval action), Ledger, Configuration Schema.

E • Paying a EUR invoice in USDT, in pieces

Where it stands. Wanted by NPL (8 Oct): "good to have". Vendors are paid on different days; the forex gain or loss between collection and payment is accepted, with the 2 % markup as the buffer.

WHAT IS HAPPENING

A reseller customer owes a vendor an invoice in euros. They pay NPL in THB or USDT; NPL pays the vendor in USDT, often in several instalments over days, each converted at that day's market rate (EUR/USDT was 0.8708 at collection and 0.87527 at the final payment in the 27 Group case). The question at every moment is "how much of the invoice is still unpaid?" — and the answer changes with the rate on each instalment.

WHY THE MODEL CANNOT SAY IT

The model tracks amounts in the currency they move in. A payout line knows which deal leg it settles (D17), but not how many euros of obligation that USDT payment discharged or at what rate; and the obligation itself — the invoice — has no home beyond a free-text client reference.

THE FIX, AND HOW IT CLOSSES THE GAP

Give the invoice a record of its own and each payout line two facts — "EUR discharged" and the market snapshot used. The outstanding balance becomes a sum (obligation – Σ discharged), each instalment's rate is on record, and the advance-then-balance pattern is two collections against one obligation.

ERD FOOTPRINT

- New table INVOICE: id, project_id (FK), receiver_id (FK — the vendor), currency, amount, reference, issued_at, status (open / settled / void). PROJECT owns INVOICE; RECEIVER refs INVOICE; DEAL_GROUP gains invoice_id (FK, nullable).
- DISBURSEMENT_LINE gains invoice_id (FK, nullable), obligation_discharged (decimal, invoice currency), discharge_rate (decimal) and discharge_market_rate_id (FK → MARKET_RATE).
- New view INVOICE_BALANCE(invoice) = amount – Σ obligation_discharged over confirmed lines. New invariant: Σ discharged never exceeds amount; a line naming an invoice names a deal_id whose group is on that invoice.
- Ledger: the difference between the EUR value collected and the EUR discharged posts to VARIANCE with the rounding-style cost component fx_timing, so reseller revenue and remittance margin stay apart. Diagrams 1a, 3a, 3b. Count: 56 tables, 6 views.

F · Rounding that isn't "drop the cents"

Where it stands. Confirmed by NPL (8 Oct): round up to the nearest 1,000 for Asian currencies (THB, INR, JPY ...), to the nearest unit for USD, SGD, HKD; round up the USDT a reseller customer pays "when possible" — deliberate, not a convenience.

WHAT IS HAPPENING

The agreed rule is: at conversion, cut the amount to whole units and keep the dust in the client's balance. Real quotes do other things: an INR amount is "rounded off to INR 2,329,000", THB to the nearest 1,000, a USDT amount rounded up to 2,600. These are promises made to customers.

WHY THE MODEL CANNOT SAY IT

The rounding setting on a currency pair has exactly one option: truncate to a whole unit. Cash collections can round to 100/500/1,000, but that is for cash only. There is no way to express "nearest 1,000" on an electronic payout, and no way at all to express rounding up.

THE FIX, AND HOW IT CLOSSES THE GAP

Turn the one-option setting into a small menu per currency pair — truncate to unit, nearest unit, round up to unit, nearest N (with N stated), round up to N — with the dust rule unchanged: whatever is dropped stays client money; whatever is added by rounding up is NPL's cost and is posted as such. Every live rounding practice becomes a configured, auditable rule, and the ledger still balances because the rounding difference always has an owner.

ERD FOOTPRINT

- CURRENCY_PAIR.amount_rounding grows from one allowed value to truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n, and the pair gains rounding_unit (decimal, required for the _n options, e.g. 1,000 for THB and INR). cash_rounding_unit stays for cash collections. Default by currency class: round_up_n 1,000 for Asian currencies, nearest_unit for USD, SGD, HKD.
- CONVERSION.rounding_residual becomes signed. Positive (truncation, nearest-down) stays client money; negative (round-up, nearest-up) is NPL's cost, posted with the existing rounding cost component to a new account purpose EXP_ROUNDING (LEDGER_ACCOUNT.purpose gains one value).
- Calc spec: the rounding step takes the pair's option and unit; new reference vectors (THB round-up 1,000, INR 2,329,000, USDT 2,598 → 2,600). Diagrams 1a, 4.

G · NPL's markup shared with the customer who provides the sub-account — several parties, money held by the party

Where it stands. New (8 Oct). Rule from NPL: the markup to share is the sender's fee less the partner's fee ($4\% - 1\% = 3\%$ via Ali, $4\% - 0.8\% = 3.2\%$ via Aquanow), split equally with Raeen; the partner's rebate is NPL's alone and not visible to Raeen; future deals may have more than one party with different shares; Raeen keeps the whole converted EUR in its Aquanow account and pays NPL its share monthly.

WHAT IS HAPPENING

A Raeen downline funds a Betfair sub-account that Raeen provides. The sender pays 4 %. Whatever is left after the partner's fee is split between NPL and Raeen, half each — 1.5 % each via Ali, 1.6 % each via Aquanow. Aquanow also pays NPL 0.3 % back monthly; that is NPL's and Raeen never sees it. The converted euros land in Raeen's own Aquanow account, Raeen allocates them to the sub-account and keeps its share; NPL's half is paid to NPL once a month. Tomorrow the same arrangement may involve two or three parties with different percentages.

WHY THE MODEL CANNOT SAY IT

The model has an introducer paid "a share of NPL's earnings" or "a fixed % per transaction", with the accrual posted as a payable that NPL pays out. Three things are missing: the share base is not defined as "fee less partner cost" (so a route change from Ali to Aquanow would need a reconfiguration instead of following automatically); nothing says the party, not NPL, holds the money and remits the remainder; and nothing stops the rebate being counted into the base that is shared.

THE FIX, AND HOW IT CLOSSES THE GAP

Define the share base as the net markup — sender fee less partner cost, on amount $\times$ market, rebates excluded — and let several parties hold rules on one project with their own percentages ($\Sigma \leq 100\%$). Add a settlement mode: NPL pays the party, or the party holds the converted funds and pays NPL the remainder monthly, in which case NPL's half is a receivable from that party, reconciled monthly like a partner rebate. The route change then needs no reconfiguration: the engine takes the partner cost from whichever partner converted.

ERD FOOTPRINT

- `REFERRAL_RULE.basis` gains the value `share_of_net_markup` (`base = fee_sender_part + fee_receiver_part - partner_cost`, on the deal; never includes `PARTNER_REBATE_ACCRUAL`). `REFERRAL_RULE` gains `settlement_mode` (`npl_pays / party_retains`) and `party_kind` (`introducer / client_party`). Several rules per project with different `pct` are already allowed; new rule: $\Sigma \text{ pct of share_of_net_markup rules active on a project} \times \text{partner-pair} \leq 100\%$.
- `REFERRAL_ACCRUAL` gains `direction` (`payable / receivable`), `period` (`YYYY-MM`) and `received_amount`, with states `accrued / reconciled / settled`, so a `party_retains` rule reconciles monthly against the party's remittance exactly as a rebate does against a partner statement.
- `LEDGER_ACCOUNT.purpose` gains `SHARE_RECEIVABLE` (`CO.<INTRODUCER>.<CCY>.SHARE_RECEIVABLE`), `holder_type` `Introducer`. Postings for `party_retains`: at conversion NPL's earnings are recognised in full, the party's share is accrued (`earnings  $\rightarrow$  payable`), and the two are netted into one receivable from the party for NPL's remainder; the pattern is worked in the Ledger Posting Design.
- Roles: the party's share and the rebate are ECON-group fields (Finance and above); a party-facing statement shows the share only, never the rebate. Diagrams 1b (`REFERRAL_RULE`), 3b (`accrual`), 4 (`purpose`). Dependent docs: Ledger (new pattern), Configuration Schema, Roles matrix.

H · Rebates as a separate earnings stream — and statement lines for deals NPL never saw

Where it stands. New (8 Oct). From NPL: rebates must be tracked separately when NPL's earnings are aggregated. Aquanow and Jeton also pay rebates on other clients NPL referred but does not service, under the

same rebate logic, so the rebate NPL can predict from its own deals is only a subset of what the partner's statement will show.

WHAT IS HAPPENING

Jeton and Aquanow take their full fee on every deal and pay NPL a slice back once a month — 0.4 % of 1 % at Jeton, 0.3 % of 0.8 % at Aquanow. For BF deals via Jeton that slice is NPL's only income. The monthly statement from the partner also carries rebates for clients NPL introduced to the partner but never handles: no quote, no collection, no deal in NPLify. Management wants to see "earnings on deals" and "rebates" as two numbers, never one.

WHY THE MODEL CANNOT SAY IT

The model has the rebate rule and a per-deal expected accrual reconciled monthly, which is right for serviced deals. But every accrual needs a `deal_id`, so a statement line with no deal behind it has nowhere to go but "variance"; and the chart of accounts books rebate income into the same stream as deal earnings (`EARN_GROSS`), so the two cannot be reported apart without a manual split.

THE FIX, AND HOW IT CLOSSES THE GAP

Give rebate income its own purpose in the chart of accounts and keep it out of every "earnings" total by construction. Let a rebate accrual exist without a deal, tagged by origin — a serviced deal, or a referred client NPL does not service — so the monthly reconciliation is: statement total = Σ expected on serviced deals + referred-client rebates, with only the remainder being a variance to investigate. The Partner Commissions report shows the three figures.

ERD FOOTPRINT

- `LEDGER_ACCOUNT.purpose` gains `EARN_REBATE` (income, holder Project or None); `REBATE_RECEIVABLE` stays as the asset. Rebate postings never touch `EARN_GROSS`; the earnings views and the P&L report sum `EARN_GROSS` and `EARN_REBATE` as separate lines.
- `PARTNER_REBATE_ACCRUAL.deal_id` becomes nullable; the table gains `origin` (`serviced_deal` / `referred_client`) and `statement_ref` (string). A `referred_client` accrual is created from the partner statement at reconciliation, with `expected_amount` null and `received_amount` set; a `serviced_deal` accrual keeps today's shape. Invariant: `deal_id` is required when `origin = serviced_deal` and must be null otherwise.
- `PARTNER_REBATE_RULE` gains `covers_referred_clients` (bool) so the reconciliation knows whether to expect statement lines beyond NPL's deals for this partner. Optional later: a `PARTNER_STATEMENT` header (already on the backlog) to hold the statement total and period.
- Roles: rebate figures stay in the ECON group; the referred-client total is Finance and Management only. Diagrams 3b (accrual), 4 (purpose). Dependent docs: Ledger (rebate pattern restated), Roles matrix, Configuration Schema.

I · Partner rates quoted the other way round

Where it stands. New (8 Oct). From NPL: Ali sometimes gives USD → EUR as 1.1738 (EUR per USD the other way round), "so we have to reverse it"; $1 \div 1.1738 = 0.851934$ confirmed as Ali's rate on the BF-Seven example.

WHAT IS HAPPENING

NPL writes every rate as "units received per 1 unit sent" (0.8519 EUR per USD). Ali, on some days, states the same price as 1.1738 USD per EUR. Operations invert it by hand before quoting. One slip — entering 1.1738 where 0.8519 was meant — would price a deal 38 % wrong.

WHY THE MODEL CANNOT SAY IT

PARTNER_RATE_VERSION. rate is a single decimal assumed to be in the pair's quote_direction. There is no record of how the partner stated it, so a wrong-direction entry is undetectable, and the audit trail shows a number the partner never sent.

THE FIX, AND HOW IT CLOSSES THE GAP

Record the rate as the partner stated it and in which direction, and store the normalised rate beside it. The engine inverts when the directions differ, at the pair's precision, and a sanity check rejects a normalised rate that is more than a configurable distance from the market snapshot — which is exactly the case where a direction slip would land.

ERD FOOTPRINT

- PARTNER_RATE_VERSION gains quoted_rate (decimal, as stated by the partner) and quoted_direction (out_per_in / in_per_out). rate stays the normalised value in CURRENCY_PAIR.quote_direction; rule: $\text{rate} = \text{quoted_rate}$ when directions match, else $1 \div \text{quoted_rate}$ rounded to rate_precision. rate is derived, never typed.
- Reuses the existing spread check: a normalised rate whose spread_pct against the market snapshot exceeds the breach threshold goes to breached — a direction slip is caught the same way an off-market rate is.
- Calc spec: one acceptance vector (1.1738 → 0.851934 at 6 decimals). Diagram 2 (rates). Dependent docs: Calculation Specification, Configuration Schema (per-pair precision).

J · Bank cutoff, rate lock and next-day conversion

Where it stands. New (8 Oct). From NPL's Project Context: for Evo via Ali the rate is compared in the morning; anything locked with the partner before 14:30 GMT+8 converts the same day, otherwise the collection goes ahead, the sender's calculation keeps the morning's figures, and the conversion happens the next business day at that day's partner rate. AK's USD and SGD balances are "banking-ready" and wait for the cutoff too. Open: who bears the difference between the morning rate and the next day's rate.

WHAT IS HAPPENING

Any currency pair that depends on a banking channel has a daily cutoff. Before it, a deal locked with the partner converts today at today's rate even if the money arrives later in the day. After it, the money is still collected and the sender is told the morning's numbers, but the partner only converts tomorrow morning — at tomorrow's rate. The same applies to balance conversions into USD or SGD at Ali.

WHY THE MODEL CANNOT SAY IT

The model has quote freshness, quote validity and a grace window, all measured from the quote. It has no notion of a partner's daily cutoff, of a rate being locked with the partner (as distinct from quoted to the sender), or of a conversion that is due on a later business day at a rate not yet known. The lifecycle goes Collected → Converting with nothing in between.

THE FIX, AND HOW IT CLOSSES THE GAP

Put the cutoff on the partner configuration and flag the pairs that depend on the banking channel. Stamp on the leg when the rate was locked with the partner and the business day the conversion is due. A leg collected after the cutoff waits in "Collected — awaiting cutoff" and converts at the next day's partner rate version; the sender's figures are unchanged (they are write-once), and the difference between the stamped partner rate and the actual one posts to variance for Finance to see — or the leg is re-quoted, if NPL decides the sender bears it.

ERD FOOTPRINT

- PARTNER_CONFIG gains bank_cutoff_time (time) and cutoff_timezone (e.g. Asia/Hong_Kong). PARTNER_PAIR gains needs_bank_lock (bool): true for pairs whose conversion depends on the banking channel (all Evo pairs into EUR; AK's USDT → USD / SGD).

- DEAL gains `rate_locked_at` (datetime, when the partner confirmed the rate) and `conversion_due_date` (date). Rule: `conversion_due_date` = the lock date when `rate_locked_at` < cutoff, else the next business day; a leg may not enter Converting before its due date. `BALANCE_CONVERSION` gets the same two columns.
- Lifecycle: a sub-state of Collected, "awaiting cutoff", with the due date on the daily exception list if it passes unconverted. Variance between `partner_rate_version_id` stamped at quote and the version used at conversion posts to `VARIANCE` with cost component `cutoff_timing` — or the leg goes to Requested under the project's `late_collection_policy`, once NPL answers who bears it.
- Diagram 3a (DEAL columns), 1b (PARTNER_CONFIG). Dependent docs: Lifecycle (new sub-state), Configuration Schema, Calculation Specification (business-day calendar per partner).

K · Cash by token, with rates that depend on the city

Where it stands. New (8 Oct). From NPL's Payment methods tab: a cash collection or settlement is arranged as a "token" — a local-currency bill whose serial number identifies the agent; the sender photographs the token as proof, the partner counts the cash and confirms. Rates differ by location (INR in Delhi vs Mumbai, USD cash in Bangkok vs Singapore) and the location must be known before a token can be arranged.

WHAT IS HAPPENING

Cash never touches a bank account. NPL gets a token number and the agent's contact from the partner, the sender meets the agent, hands over the cash and receives the token; a photo of the token closes the loop and the partner's count confirms the amount. Every step is evidence that today lives in chat threads. And a cash rate is not one number per currency: Delhi and Mumbai are two different agents with two different rates.

WHY THE MODEL CANNOT SAY IT

`DEAL.cash_instructions` is free text and `COLLECTION.evidence_ref` one reference; there is no place for the token, the agent, the hand-over step or the location, and `PARTNER_RATE_VERSION` is per partner × pair × day with no location, so two cities on the same day cannot both be recorded.

THE FIX, AND HOW IT CLOSSES THE GAP

Give cash legs a location, and cash partner rates a location too, so "INR cash, Delhi" and "INR cash, Mumbai" are two rate versions on the same day. Record the token and the agent on the collection (and on the settlement line for cash payouts), with the token photo as evidence and the partner's count as the verification step. The collection lifecycle for cash then has named steps instead of one "verified".

ERD FOOTPRINT

- DEAL gains `cash_location` (string, required when `collection_method` = cash or a cash payout is planned). `PARTNER_RATE_VERSION` gains `location` (string, nullable; required for cash pairs) and the uniqueness of a rate version becomes partner × pair × day × location.
- `COLLECTION` gains `token_ref` (the bill's serial), `agent_contact_ref` (external store) and `counted_at` (datetime, the partner's count); `evidence_ref` holds the token photo. `DISBURSEMENT_LINE` gains `token_ref` for cash settlements. Collection states for cash: `token_issued` → `handed_over` (photo) → `counted` (partner) → `verified`.
- Roles: the agent's contact details are OPS-visible; nothing economic changes. Diagrams 2 (rate version), 3a (DEAL), 3b (COLLECTION, line). Dependent docs: Lifecycle (cash sub-states), Configuration Schema (locations as reference data).

L · Sender wallet screening and risk-routed receiving wallets

Where it stands. New (8 Oct). From NPL's Payment methods tab: the sender declares a sending wallet; NPL screens it; acceptable wallets are given the partner's standard receiving wallet, higher-risk-but-acceptable wallets an alternate receiving wallet, unacceptable ones are refused; the sender shares the transaction hash, NPL verifies it on chain, the partner confirms. Receivers' wallets are screened the same way before a settlement. Not all Evo senders accept the up-front screening.

WHAT IS HAPPENING

Before a crypto collection, NPL decides whether it will accept money from that wallet at all, and if so into which of the partner's wallets it should land. That decision is a compliance record: who screened, what the result was, which wallet was then given. It also explains why two senders of the same client pay into different partner wallets.

WHY THE MODEL CANNOT SAY IT

SENDER.risk_tier is a static label on the sender, not a record of a screening of a particular wallet on a particular day; COLLECTION does not record the sending address at all; and nothing ties the endpoint chosen to the screening result, so the rule "elevated-risk wallets go to the alternate endpoint" cannot be checked.

THE FIX, AND HOW IT CLOSSES THE GAP

A screening record per wallet address — subject, result, who, when, decision — referenced by the collection (or the settlement registration for a receiver's wallet). Each collection endpoint carries a risk band, and a rule ties them: a collection from a wallet whose screening decision is "accept, alternate" may only land on an elevated-band endpoint; one with "reject" may not be created. Screening stays optional per project (Evo), in which case the collection records that it was skipped.

ERD FOOTPRINT

- New table WALLET_SCREENING: id, project_id (FK), subject_kind (sender_wallet / receiver_wallet), sender_id / receiving_entity_account_id (FK, one of), network, address, provider_ref, result (standard / elevated / unacceptable), decision (accept / accept_alternate / reject), screened_by (FK), screened_at, evidence_ref. PROJECT owns it; SENDER and RECEIVING_ENTITY_ACCOUNT ref it.
- COLLECTION gains sending_address (string) and wallet_screening_id (FK, nullable) with screening_skipped (bool) for projects that do not screen up front; evidence_ref keeps the transaction hash and verified_at the on-chain check. COLLECTION_RECEIVING_ENDPOINT gains risk_band (standard / elevated).
- New invariant: a COLLECTION whose screening decision is accept_alternate references an endpoint with risk_band = elevated; one whose decision is reject cannot exist. SETTLEMENT_REGISTRATION for a wallet account may reference the receiver-side screening.
- Roles: screening results are a CTRL-group field (Finance and Management; Operations sees the decision only). Diagrams 1a (new box), 3b (COLLECTION), 1b (endpoint). Dependent docs: Lifecycle (screening before Collecting), Roles matrix. Count: +1 table.

M · One-time local bank accounts and the reversal of a collection

Where it stands. New (8 Oct). From NPL's Payment methods tab: local-currency deposits use ad hoc accounts the partner provides for one transaction and a specific amount; if the receiving account is frozen and the funds are lost because of the sender's side, that is a "reversal of collection" borne by the sender. Local payouts for small amounts (SG, HK, Indonesia, Thailand) are made by cash deposit or local transfer.

WHAT IS HAPPENING

For an INR or other local-currency deposit the partner opens or lends a bank account for exactly one transaction and one amount; the sender deposits, shares the slip, and the account is not used again. Occasionally the bank freezes that account after the deposit; if the cause is on the sender's side, the money is gone for the sender and the collection is undone.

WHY THE MODEL CANNOT SAY IT

A `COLLECTION_RECEIVING_ENDPOINT` is a standing account: active until retired, usable by any number of collections, with no amount or deal attached. And the lifecycle has no event by which a verified collection is reversed because the funds were lost at the partner's bank; the only exits after `Collected` are `conversion`, `reroute` or a `void` with `Management` disposition.

THE FIX, AND HOW IT CLOSES THE GAP

Mark an endpoint as `single-use` and reserve it for one deal and one expected amount; after its collection is verified it retires itself. Add a collection reversal as a named exception: the collection moves to `Reversed`, a reversing ledger transaction takes the amount out of the client's balance, the loss is attributed per `Management`'s decision (sender by default), and the deal leg goes back to `Collecting` or is voided.

ERD FOOTPRINT

- `COLLECTION_RECEIVING_ENDPOINT` gains `usage` (`standing / single_use`), `reserved_for_deal_id` (FK, nullable) and `expected_amount` (decimal, nullable). Rule: a `single_use` endpoint is referenced by at most one `COLLECTION`, must match the reserved deal, and moves to `retired` when that collection is verified.
- `COLLECTION.state` gains `reversed`; new table-less event: `LEDGER_TRANSACTION.source_type` gains `CollectionReversal`, posting a reversing entry against `CL.<PARTNER>.<CCY>.COLLECTED` with a `loss_bearer` (`sender / npl / partner`) recorded on the collection (`reversal_reason`, `reversed_at`, `loss_bearer`). `Management` approval: `approve_collection_reversal`.
- Lifecycle: `Collected` → `Reversed` (account frozen) → leg returns to `Collecting` or is `Voided` after collection with disposition. Invariant 4 (client balance never negative) is preserved because the reversal is capped at the verified amount.
- Diagrams 1b (endpoint), 3b (`COLLECTION`), 4 (source type). Dependent docs: Lifecycle (exception path and approval), Ledger (reversal pattern), Configuration Schema.

3 · Facts from NPL's tabs that need no new structure

Fact	How the v5.2 model holds it
A transfer inside the partner's platform	Aquanow pays BF by an internal transfer within the Aquanow platform — instant, no bank. <code>RECEIVING_ENTITY_ACCOUNT.rail</code> and <code>COLLECTION_RECEIVING_ENDPOINT.rail</code> accept the value <code>internal</code> (account held at the same partner); <code>SETTLEMENT_RAIL</code> for it has no bank fee. D18's nullable rail already allows it; only the value list grows.
Evo's two receiver groups	Bplay (Babylon, Bplay, Lex) and TBet (TBet), senders designated per group, Ali able to pay Babylon and TBet, Jeton able to pay Bplay, Lex and TBet. This is exactly <code>RECEIVER_GROUP + SENDER_RECEIVER_ALLOW + SETTLEMENT_REGISTRATION</code> per partner. Configuration, not a change.
AK Personal	A stakeholder balance at Ali up to about USD 400,000, no markup, withdrawals in several forms. <code>PROJECT.disburse_policy = hold_allowed</code> with <code>exposure_ceiling</code> , a 0 % fee structure with 0 % minimum margin, and <code>BALANCE_CONVERSION</code> for the USD / SGD switches. Cash withdrawals use K; the cutoff uses J.
Peru remittance	Bank transfer to Ali's receiving entity (PT Silver Lining Production), USD → USDT, Ali 3 % + NPL 1 %. A standard disclosed-rate pattern with a bank-in endpoint.
Narratives	The name on a payout is often the customer's own entity (Seven Investments N.V., Bplay) or NPL's own entity (New Pinnacle Ltd for GDC, NPL for 66 Group, and occasionally as the declared sender for YN Remit). <code>SETTLEMENT_SENDING_ENTITY</code> already models this; the Understanding should say that NPL's entities are sending entities too.
Jeton inactive; Aquanow preferred for BF	<code>PARTNER_CONFIG.status</code> and <code>PARTNER_PAIR.status</code> carry it; the quoting-rate policy picks the active partner. Novi's 0.9 % at Jeton is a per-sender <code>FEE_OVERRIDE</code> .
JTN-JPY cash, YN Remit, 66 Group multi-currency	Partner rate + a variable markup within a maximum (JTN: 0.2–0.5 %, max 0.5 %; YN: partner + 0.5 %): <code>FEE_STRUCTURE.mode = variable</code> with <code>floor_pct / cap_pct</code> — NPL calls them lowest and highest markup, and the NPL-facing documents now use those words. Sender = receiver is allowed (same party in both roles).
Collect-first vs quote-first senders	Evo senders who "pay what they can" are <code>entry_mode = collect_first</code> ; those who pay the exact invoice are <code>quote_first</code> . Per sender, configuration.

4 · Footprint of all thirteen, and what is Management's to decide

#	Proposal	New tables	Changed tables	Rules / invariants	Views	Diagrams	Decision level
A	A special sender gets a different split	—	FEE_OVERRIDE (+2), DEAL (+1)	1 rule	—	1b, 3a	Additive — New XP with NPL's parameters
B	Fees that drop with volume	FEE_TIER	DEAL (+2)	1 new	+1 (month volume)	1b, 3a	Additive — New XP with NPL's parameters
C	The fee the customer sees isn't the fee that was charged	—	FEE_STRUCTURE (+2), DEAL_GROUP (+2)	1 rule	—	1b, 3a	Additive — New XP with NPL's parameters
D	NPL as its own exchange desk — NPL-GR as a project	—	PARTNER_CONFIG (+1), COLLECTION_RECEIVING_ENDPOINT (+1)	invariant 3 restated; 1 approval action	CUSTODY extended	1a, 1b, 4	Management — changes a principle (no custody)
E	Paying a EUR invoice in USDT, in pieces	INVOICE	DEAL_GROUP (+1), DISBURSEMENT_LINE (+4)	1 new	+1 (invoice balance)	1a, 3a, 3b	Additive — New XP with NPL's parameters
F	Rounding that isn't "drop the cents"	—	CURRENCY_PAIR (+1 value set, +1 column), LEDGER_ACCOUNT purpose (+1)	—	—	1a, 4	Additive — New XP with NPL's parameters
G	NPL's markup shared with the customer who provides the sub-account — several parties, money held by the party	—	REFERRAL_RULE (+2, +1 value), REFERRAL_ACCRUAL (+3), LEDGER_ACCOUNT purpose (+1)	1 new (Σ shares $\leq$ 100 %)	—	1b, 3b, 4	Finance — reporting and reconciliation shape

#	Proposal	New tables	Changed tables	Rules / invariants	Views	Diagrams	Decision level
H	Rebates as a separate earnings stream — and statement lines for deals NPL never saw	—	PARTNER_REBATE_ACCRUAL (+2, deal_id nullable), PARTNER_REBATE_RULE (+1), LEDGER_ACCOUNT purpose (+1)	1 new	earnings views split	3b, 4	Finance — reporting and reconciliation shape
I	Partner rates quoted the other way round	—	PARTNER_RATE_VERSION (+2)	1 rule	—	2	Additive — New XP with NPL's parameters
J	Bank cutoff, rate lock and next-day conversion	—	PARTNER_CONFIG (+2), PARTNER_PAIR (+1), DEAL (+2), BALANCE_CONVERSION (+2)	1 rule	—	1b, 3a	Additive — New XP with NPL's parameters
K	Cash by token, with rates that depend on the city	—	DEAL (+1), PARTNER_RATE_VERSION (+1), COLLECTION (+3), DISBURSEMENT_LINE (+1)	1 rule (uniqueness)	—	2, 3a, 3b	Additive — New XP with NPL's parameters
L	Sender wallet screening and risk-routed receiving wallets	WALLET_SCREENING	COLLECTION (+3), COLLECTION_RECEIVING_ENDPOINT (+1)	1 new	—	1a, 1b, 3b	Additive — New XP with NPL's parameters
M	One-time local bank accounts and the reversal of a collection	—	COLLECTION_RECEIVING_ENDPOINT (+3), COLLECTION (+3, +1 state), LEDGER_TRANSACTION source (+1)	1 rule	—	1b, 3b, 4	Finance — adds an approval (collection reversal)

Net, if all thirteen go in: **54 → 57 tables** (FEE_TIER, INVOICE, WALLET_SCREENING), **4 → 6 views** (month volume, invoice balance) plus the earnings views split by stream, **19 → 27 invariants or rules** with invariant 3 restated, three new account purposes (EXP_ROUNDING, SHARE_RECEIVABLE, EARN_REBATE), one new transaction source (CollectionReversal), two new approval actions (enable_own_desk, approve_collection_reversal), about 45 new columns, and no table removed. D is the only proposal that changes a principle rather than adding structure; G and H change what Finance and Management see in the earnings reports and are worth their explicit nod; the other ten are additive and can ship as v5.3 together.

5 · Dependent documents touched by v5.3

ERD baseline (all), Configuration Schema (A, B, C, D, F, G, H, I, J, K, M), Calculation Specification (A, C, F, I, J), Ledger Posting Design (D, E, F, G, H, M), Lifecycle & Approvals (D, J, K, L, M), Roles & Visibility Matrix (C, G, H, L), ERD Reading Guide (every diagram that gains a box), Fee Outlines (the "needs a system change" notes become "configured").

6 · Questions still open with NPL

Consolidated from the "Please confirm" lists of Fee Outlines v0.2; the first column names the proposal the answer feeds.

Feeds	Question
B	Does the lower GDC tier apply to the transaction that takes the month past USD 500,000, or only from the next one? Is the threshold counted for GDC as a whole?
G	Raeen via Aquanow: the Project Context example deducts 3 % before converting; under the agreed rule it should be 3.2 %. Confirm 3.2 %. Row 14 (direct downlines): 0.2 % shared 75/25 — confirm 0.15 % to Raeen, 0.05 % to NPL.
J	After the 14:30 cutoff, who bears the difference between the morning's partner rate (used on the sender's calculation) and the next day's actual rate — NPL as a variance, or the sender via a re-quote?
D	Management: run NPL-GR inside NPLify as an own-desk project (NPL's suggestion), or keep the no-custody principle and run the reseller flows outside the system?
F / K	INR cash is rounded to the nearest 1,000 on the collected amount (2,328,878 → 2,329,000): is that "round up" like THB, or "nearest"? Cash rounding units for INR and for AED / USD cash in Dubai.
—	Ad Hoc INR Cash: 7 % (entered on the sheet) or 0.5 % (fee table row 45)? And is it NPL's fee on top of the agent's rate, or the agent's own?
—	Po8: should the system refuse a THB collection by itself when Ali's rate is more than 1.5 % below market, or only warn Finance? Is there a limit on accepted forex gain / loss across vendor payments beyond the 2 % buffer?
—	BF withdrawals: Jeton only, or can Ali receive them and at what fee? Novi at 0.9 % via Jeton: still 0.4 % back, or 0.3 %?
—	Same-currency pass-through (Evolution bank transfers): still in scope?
C	When the BF displayed-fee rule changes to "partner rate + markup", tell us — the displayed-fee setting is simply cleared.

Draft v1.0 — internal. On approval, each proposal is recorded as a decision (D20 onward) in the decision record, applied to data/erd.v5.json, the map and the knowledge base, and the ERD baseline and dependent documents are regenerated as v5.3.